Efficient Python

Lecture 03

CERN
STEAM Academy

08 July 2026

Numerical Foundations for Machine Learning

Jacek Gatlik

email: jacek.gatlik@agh.edu.pl
www: https://galaxy.agh.edu.pl/~jgatlik/

Faculty of Physics and Applied Computer Science,
AGH University of Krakow, Poland

AGH

Outline

Data, models, and losses

Gradients and iterative training

Numerical stability, validation, and scientific use

Further reading

Section 1

Data, models, and losses

From numerical experiments to learning workflows

In the previous lectures, we treated arrays as computational versions of vectors, matrices, tensors, sampled fields, parameter sweeps, and diagnostics. Machine learning uses the same numerical language, but the object being adjusted is now a model with parameters.

$$\text{data} \longrightarrow \text{model} \longrightarrow \text{residual} \longrightarrow \text{loss} \longrightarrow \text{gradient} \longrightarrow \text{updated parameters}$$

This lecture is not a catalogue of machine learning algorithms. Its goal is to make the numerical structure behind machine learning workflows transparent.

The same habits remain important: inspect shapes, understand axes, control scales, and treat numerical diagnostics seriously.

Machine learning as model fitting

In many scientific problems, we already fit models to data. Machine learning can often be viewed as a more flexible version of the same idea.

$$y_i \approx f_\theta(x_i), \qquad \theta^\star = \arg\min_\theta L(\theta).$$

Here x_i denotes the input data, y_i denotes the target or observation, f_θ is a parameterized model, and $L(\theta)$ measures how badly the model disagrees with the data.

For scientist, this should feel familiar. We choose a parameterized description, compare it with observations or simulations, define an objective function, and use a numerical method to improve the parameters.

Data arrays and axis conventions

A supervised learning dataset is usually stored as an array of inputs and an array of targets. The most common convention is:

$$X_{ij} = \text{feature } j \text{ of sample } i.$$

```
n_samples = 1000
n_features = 4

X = np.zeros((n_samples, n_features))
y = np.zeros(n_samples)

X.shape    # (1000, 4)
y.shape    # (1000,)
```

Axis 0 labels samples. Axis 1 labels features. This convention is not only technical; it determines how we compute predictions, losses, gradients, batches, and diagnostics.

Samples, features, and targets

A feature does not have to be a spatial coordinate. It may be a time, a parameter value, a measured intensity, a local field value, a detector response, or a quantity extracted from a simulation.

```
rng = np.random.default_rng(123)

time = np.linspace(0.0, 10.0, 500)
field = rng.uniform(-1.0, 1.0, size=time.size)
temperature = 300.0 + 5.0*rng.normal(size=time.size)

X = np.column_stack([time, field, temperature])
y = np.sin(time) + 0.2*field
```

In this example, every row of `X` describes one observation. The target `y` is the quantity we want the model to reproduce or predict.

Synthetic data with known ground truth

Synthetic data are useful because we know what structure should be recovered. They allow us to test the whole numerical workflow before using real data.

```
rng = np.random.default_rng(123)

N = 500
x = rng.uniform(-2.0, 2.0, size=N)

theta_true = 1.5
bias_true = -0.7
noise = 0.2 * rng.normal(size=N)

y = theta_true*x + bias_true + noise
X = x[:, None]

X.shape    # (500, 1)
y.shape    # (500,)
```

This already contains the essential ingredients: inputs, targets, noise, unknown parameters, and a model structure that we hope to recover.

Splitting data is part of the experiment

A model should not only fit the data it has already seen. We usually split the data into training, validation, and test parts.

```
rng = np.random.default_rng(123)

idx = rng.permutation(N)

n_train = int(0.7*N)
n_val = int(0.15*N)

train_idx = idx[:n_train]
val_idx = idx[n_train:n_train+n_val]
test_idx = idx[n_train+n_val:]

X_train, y_train = X[train_idx], y[train_idx]
X_val, y_val = X[val_idx], y[val_idx]
X_test, y_test = X[test_idx], y[test_idx]
```

The training set is used to adjust parameters. The validation set is used to monitor choices such as model complexity or learning rate. The test set should remain untouched until the final evaluation.

Data leakage and preprocessing

Preprocessing must respect the split between training and validation or test data. Otherwise, information from the validation or test set can accidentally influence the training procedure.

```
mu = X_train.mean(axis=0)
scale = X_train.std(axis=0)

scale[scale == 0.0] = 1.0

X_train_s = (X_train - mu) / scale
X_val_s = (X_val - mu) / scale
X_test_s = (X_test - mu) / scale
```

The mean and scale are computed from the training data only. They are then applied to all other subsets. This is a small detail, but it is essential for honest validation.

A model is a parameterized numerical map

A model turns an input array into a prediction array. In the simplest case, the model is linear in the features:

$$\hat{y} = X\theta + b.$$

```
def linear_model(X, theta, bias):
    return X @ theta + bias
```

If `X.shape == (N, d)` and `theta.shape == (d,)`, then the prediction has shape `(N,)`. The sample axis survives; the feature axis is contracted with the parameter vector.
The model is therefore not an abstract black box. It is a numerical map with specific input and output shapes.

Linear model as matrix computation

The formula

$$\hat{y}_i = \sum_{j=1}^{d} X_{ij}\theta_j + b$$

is exactly a matrix-vector multiplication followed by adding a scalar bias.

```
n_features = X_train_s.shape[1]

theta = np.zeros(n_features)
bias = 0.0

y_pred = X_train_s @ theta + bias

y_pred.shape    # (n_train,)
```

This is a useful first mental model for many learning systems: predictions are computed by composing array operations.

Batched predictions

Sometimes we want to evaluate many candidate parameter vectors at once. We can add an axis for the candidate model.

```
n_models = 100

Theta = rng.normal(size=(n_models, X_train_s.shape[1]))
biases = rng.normal(size=n_models)

Y_pred = X_train_s @ Theta.T + biases[None, :]

Y_pred.shape    # (n_train, n_models)
```

The first axis labels samples. The second axis labels candidate models. This is the same array thinking as in parameter sweeps or batched linear algebra.

Parameters are arrays

In a linear model, the parameters may be one vector and one scalar. In a neural network, the parameters are usually several arrays: matrices and bias vectors.

```
n_features = 3
n_hidden = 16
n_outputs = 1

params = {
    "W1": np.zeros((n_features, n_hidden)),
    "b1": np.zeros(n_hidden),
    "W2": np.zeros((n_hidden, n_outputs)),
    "b2": np.zeros(n_outputs),
}
```

The parameter set is more complicated, but the same principle remains: every parameter has a shape, and that shape must be compatible with the model computation.

A nonlinear model as a composition

A simple neural network is a composition of linear maps and elementwise nonlinear functions.

```python
def relu(z):
    return np.maximum(z, 0.0)

def network(X, params):
    h = relu(X @ params["W1"] + params["b1"])
    y = h @ params["W2"] + params["b2"]
    return y.squeeze()
```

The expression is still built from familiar operations: matrix multiplication, broadcasting, and elementwise functions. The model becomes flexible because the intermediate representation `h` is learned from data through the parameters.

Activation functions as elementwise maps

Activation functions are nonlinear maps applied element by element. They are one reason why a neural network is not equivalent to one large linear transformation.

$$\tanh z, \qquad \sigma(z) = \frac{1}{1+\exp(-z)}, \qquad \mathrm{ReLU}(z) = \max(0, z).$$

```
z = np.linspace(-5.0, 5.0, 1000)

a_tanh = np.tanh(z)
a_sigmoid = 1.0 / (1.0 + np.exp(-z))
a_relu = np.maximum(z, 0.0)
```

Numerically, these are just array operations. Conceptually, they control what kind of nonlinear dependence the model can represent.

Residuals are arrays

The residual compares prediction and target sample by sample.

$$r_i = \hat{y}_i - y_i.$$

```
y_pred = linear_model(X_train_s, theta, bias)

residual = y_pred - y_train

rms_residual = np.sqrt(np.mean(residual**2))
max_residual = np.max(np.abs(residual))
```

The residual array contains local information about the errors. A single scalar diagnostic is useful, but it hides where and how the model fails.

Loss functions reduce residuals

A loss function reduces residual information to a scalar objective. For regression, the mean squared error is one of the simplest choices:

$$L(\theta, b) = \frac{1}{2N} \sum_{i=1}^{N} \left(\hat{y}_i - y_i\right)^2 .$$

```python
def mse_loss(y_pred, y):
    residual = y_pred - y
    return 0.5 * np.mean(residual**2)

loss = mse_loss(y_pred, y_train)
```

The factor $1/2$ is not essential, but it makes derivatives slightly cleaner.

Weighted losses and experimental uncertainty

In experimental physics, not all data points necessarily have the same uncertainty. A weighted loss can reflect known measurement errors.

$$L = \frac{1}{2N} \sum_{i=1}^{N} \left(\frac{\hat{y}_i - y_i}{\sigma_i} \right)^2 .$$

```
sigma = 0.1 + 0.05*np.abs(y_train)

residual = y_pred - y_train
weighted_loss = 0.5 * np.mean((residual / sigma)**2)
```

The choice of loss function is part of the model. It defines what kind of error the optimizer is asked to reduce.

Scaling and nondimensionalization

An optimizer sees numbers, not physical units. If one feature is of order 10^{-6} and another is of order 10^{6}, the loss landscape may become poorly conditioned.

$$\tilde{x}_j = \frac{x_j - \mu_j}{s_j}.$$

This is closely related to nondimensionalization in physics. We do not change the information content of the problem; we change the numerical representation to make the computation better behaved.

Poor scaling may slow training, amplify round-off effects, and make one learning rate inappropriate for different parameter directions.

Feature maps and basis expansions

A model that is linear in its parameters may still be nonlinear in the original input variable. This is the idea behind basis expansions and feature maps.

```
x = np.linspace(-1.0, 1.0, 500)

X_poly = np.column_stack([
    x,
    x**2,
    x**3,
])

X_fourier = np.column_stack([
    np.sin(np.pi*x),
    np.cos(np.pi*x),
    np.sin(2*np.pi*x),
    np.cos(2*np.pi*x),
])
```

Many machine learning models can be understood as learning a useful representation of the data before applying a relatively simple final map.

Section 2

Gradients and iterative training

From fitting to optimization

Once the model and loss are defined, training becomes a numerical optimization problem.

$$\theta^\star = \arg\min_\theta L(\theta).$$

In practice, we do not usually find $\theta^\star$ in one step. We construct a sequence of parameter values:

$$\theta_0, \theta_1, \theta_2, \dots$$

Each update should, at least approximately, move the parameters toward a region with smaller loss. This is a numerical method, and it has the usual numerical questions: step size, stability, convergence, diagnostics, and stopping criteria.

Gradients have the shape of parameters

For a scalar loss L, the gradient with respect to a parameter vector θ is another vector with the same shape:

$$\nabla_\theta L = \left(\frac{\partial L}{\partial \theta_1}, \ldots, \frac{\partial L}{\partial \theta_d} \right).$$

```
theta = np.zeros(n_features)
bias = 0.0

grad_theta = np.zeros_like(theta)
grad_bias = 0.0

theta.shape == grad_theta.shape    # True
```

For a neural network, every weight matrix and every bias vector has a gradient array of the same shape.

Finite differences as a gradient check

Finite differences are usually too expensive for training large models, but they are very useful for checking small gradient calculations.

$$\frac{\partial L}{\partial \theta_j} \approx \frac{L(\theta + \varepsilon e_j) - L(\theta - \varepsilon e_j)}{2\varepsilon}.$$

```python
def finite_difference_grad(loss_fn, theta, eps=1e-6):
    grad = np.zeros_like(theta)

    for j in range(theta.size):
        step = np.zeros_like(theta)
        step[j] = eps

        grad[j] = (loss_fn(theta + step)
                   - loss_fn(theta - step)) / (2*eps)

    return grad
```

This is a sanity check, not a scalable training method.

Analytical gradient for linear regression

For the linear model

$$\hat{y} = X\theta + b, \qquad r = X\theta + b - y, \qquad L = \frac{1}{2N} r^T r,$$

the gradients are

$$\nabla_\theta L = \frac{1}{N} X^T r, \qquad \frac{\partial L}{\partial b} = \frac{1}{N} \sum_{i=1}^{N} r_i.$$

```
y_pred = X_train_s @ theta + bias
residual = y_pred - y_train

N_train = residual.size

grad_theta = X_train_s.T @ residual / N_train
grad_bias = residual.mean()
```

The transpose X^T contracts the sample axis and leaves the feature axis.

Gradient descent

Gradient descent updates parameters in the direction of decreasing loss:

$$\theta_{k+1} = \theta_k - \eta \nabla_\theta L(\theta_k).$$

```
eta = 0.05

theta = theta - eta * grad_theta
bias = bias - eta * grad_bias
```

The learning rate η is a numerical step size. It is not a universal constant and it should be treated with the same care as a time step in an ODE solver.

Learning rate as a stability parameter

The learning rate controls the size of the update. If it is too small, training may be stable but very slow. If it is too large, the loss may oscillate or diverge.

$$\eta \text{ too small} \quad \Rightarrow \quad \text{slow progress,}$$

$$\eta \text{ too large} \quad \Rightarrow \quad \text{instability or divergence.}$$

A useful first diagnostic is the loss history. If the loss does not decrease at all, explodes, or becomes `NaN`, the issue may be numerical rather than conceptual.

A minimal vectorized training loop

For linear regression, the full training loop can be written using only vectorized NumPy operations.

```
theta = np.zeros(X_train_s.shape[1])
bias = 0.0
eta = 0.05
history = []
for step in range(500):
    y_pred = X_train_s @ theta + bias
    residual = y_pred - y_train
    loss = 0.5 * np.mean(residual**2)
    grad_theta = X_train_s.T @ residual / residual.size
    grad_bias = residual.mean()
    theta -= eta * grad_theta
    bias -= eta * grad_bias
    history.append(loss)
```

This loop already contains the core structure of training: forward computation, loss evaluation, gradient computation, parameter update, and monitoring.

→ **Example 03 - 01**

Monitoring the loss history

The training loop should not only update parameters. It should also record diagnostic information. The simplest diagnostic is the loss history.

```
loss_history = np.array(history)

print(loss_history[-1])
print(loss_history.min())
print(np.isfinite(loss_history).all())
```

For full-batch gradient descent, the loss should usually decrease smoothly if the learning rate is reasonable. For mini-batch training, the curve may be noisy, but the overall trend should still be meaningful.

A loss history is a numerical object. We should inspect it with the same care as any other array produced by a simulation.

Validation during training

Training loss measures how well the model fits the data used for parameter updates. Validation loss measures how well the model behaves on data not used in the update step.

```
train_history = []
val_history = []

for step in range(500):
    y_pred = X_train_s @ theta + bias
    residual = y_pred - y_train
    loss = 0.5 * np.mean(residual**2)
    grad_theta = X_train_s.T @ residual / residual.size
    grad_bias = residual.mean()
    theta -= eta * grad_theta
    bias -= eta * grad_bias
    train_history.append(loss)
    y_val_pred = X_val_s @ theta + bias
    val_history.append(mse_loss(y_val_pred, y_val))
```

Validation is not an aesthetic addition. It is one of the main tools for detecting whether the model is learning useful structure or merely adapting to the training set.

Mini-batches are slices of the sample axis

Large datasets are often not processed all at once. Instead, we use mini-batches: smaller subsets of samples used to estimate the gradient.

```
rng = np.random.default_rng(123)

N_train = X_train_s.shape[0]
batch_size = 64

batch_idx = rng.choice(
    N_train,
    size=batch_size,
    replace=False
)

X_batch = X_train_s[batch_idx]
y_batch = y_train[batch_idx]

X_batch.shape, y_batch.shape
```

The feature axis is unchanged. We only select part of the sample axis. This is another reason why a clear axis convention is so important.

Epochs and shuffling

One epoch means that the training algorithm has had the opportunity to use all training samples once. Before each epoch, it is common to shuffle the sample order.

```
n_epochs = 20
batch_size = 64
N_train = X_train_s.shape[0]

for epoch in range(n_epochs):
    idx = rng.permutation(N_train)

    for start in range(0, N_train, batch_size):
        end = start + batch_size
        batch_idx = idx[start:end]

        X_batch = X_train_s[batch_idx]
        y_batch = y_train[batch_idx]

        # compute predictions, loss, gradients, update
```

Shuffling prevents the optimizer from seeing the data in the same order every time. This matters when the dataset has an ordering, for example in time, energy, spatial position, or experimental run number.

Stochastic gradients

A mini-batch gradient is an estimate of the full gradient:

$$\nabla_\theta L_{\mathcal{B}} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \nabla_\theta \ell_i(\theta),$$

where $\mathcal{B}$ is the mini-batch and ℓ_i is the loss contribution from one sample.
This estimate is noisy, but it is often much cheaper than computing the full gradient. The noise can even help the optimizer escape some shallow or poorly conditioned regions of the loss landscape.

The price is that training curves become less smooth and diagnostics need to be interpreted statistically rather than point by point.

Full-batch, mini-batch, and online learning

Different gradient estimates correspond to different numerical compromises.

- Full-batch gradient descent uses all training samples in each update.
- Mini-batch gradient descent uses a subset of samples in each update.
- Online or stochastic gradient descent uses one sample at a time.

Full-batch updates are less noisy but may be expensive. Very small batches are cheap but noisy. Mini-batches are often a practical compromise.

From the array point of view, all three approaches differ mainly in how much of the sample axis is used to compute one update.

Closed-form fitting is not the general case

For linear regression with mean squared error, there is a classical least-squares solution. In numerical code, we should usually solve the least-squares problem directly rather than explicitly forming a matrix inverse.

```
X_aug = np.column_stack([
    X_train_s,
    np.ones(X_train_s.shape[0])
])

coef, *_ = np.linalg.lstsq(
    X_aug,
    y_train,
    rcond=None
)

theta = coef[:-1]
bias = coef[-1]
```

This is useful as a baseline. However, iterative optimization becomes necessary for nonlinear models, large models, non-quadratic losses, constraints, and many modern machine learning architectures.

Why automatic differentiation appears

For a linear model, we can derive the gradient by hand. For a deep composition of many matrix multiplications and nonlinear functions, manual differentiation becomes tedious and error-prone.

Automatic differentiation evaluates derivatives by systematically applying the chain rule to the operations used in the computation.

It is important to distinguish three ideas:

- finite differences approximate derivatives by perturbing inputs,
- symbolic differentiation manipulates formulas,
- automatic differentiation differentiates a concrete computation.

Modern machine learning frameworks use automatic differentiation to compute gradients of scalar losses with respect to large collections of parameters.

A computational graph

Consider a very small computation:

```
z = w*x + b
a = np.tanh(z)
loss = 0.5 * (a - y)**2
```

This can be viewed as a graph of operations:

$$(x, w, b) \longrightarrow z \longrightarrow a \longrightarrow L.$$

The derivative of the loss with respect to w is obtained by multiplying local derivatives along the graph:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a}\frac{\partial a}{\partial z}\frac{\partial z}{\partial w}.$$

Automatic differentiation does this bookkeeping for much larger computations.

A minimal automatic differentiation example

In a framework such as PyTorch, tensors can remember the operations used to construct them. Calling `backward()` computes gradients of a scalar loss.

```
import torch

x_t = torch.tensor(x, dtype=torch.float32)
y_t = torch.tensor(y, dtype=torch.float32)

w = torch.tensor(0.0, requires_grad=True)
b = torch.tensor(0.0, requires_grad=True)

y_pred = w*x_t + b
loss = 0.5 * torch.mean((y_pred - y_t)**2)

loss.backward()

w.grad, b.grad
```

The important concept is not the library syntax. The important concept is that the gradient is computed from the numerical operations that define the model and the loss.

The standard training step in ML frameworks

Most machine learning frameworks organize training steps around the same structure as our NumPy loop.

```
optimizer.zero_grad()

y_pred = model(X_batch)
loss = loss_fn(y_pred, y_batch)

loss.backward()
optimizer.step()
```

The names are different, but the numerical logic is the same:

$$\text{forward pass} \rightarrow \text{loss} \rightarrow \text{gradient} \rightarrow \text{parameter update}$$

Understanding the simple NumPy version makes this pattern much less mysterious.

Section 3

Numerical stability, validation, and scientific use

Training is a numerical experiment

A machine learning training run is a numerical experiment. It has inputs, parameters, numerical scales, stopping criteria, diagnostics, and failure modes.

A good training run should answer several questions:

- Are the input arrays shaped and scaled correctly?
- Is the loss finite and decreasing in a meaningful way?
- Do training and validation diagnostics agree?
- Are residuals structured or random?
- Is the model better than a simple baseline?

Without these checks, a trained model may look successful only because we have not asked sufficiently precise numerical questions.

Conditioning and scaling

For linear regression with mean squared error, the curvature of the loss is controlled by

$$H = \frac{1}{N} X^T X.$$

If the columns of X have very different scales, the eigenvalues of H may also have very different scales. Then the loss landscape becomes poorly conditioned.

In such a situation, one learning rate may be too small in one parameter direction and too large in another. Feature scaling is therefore not only a cosmetic preprocessing step; it directly affects the numerical behaviour of optimization.

Learning rate depends on numerical scale

The same physical information can be represented on very different numerical scales. The optimizer reacts to the numbers it receives, not to the units we have in mind.

```
X_bad = X_train_s.copy()

# Artificially destroy the scaling of one feature
X_bad[:, 0] *= 1.0e4

y_pred = X_bad @ theta + bias
residual = y_pred - y_train

grad_theta = X_bad.T @ residual / residual.size
```

A learning rate that works for `X_train_s` may be completely inappropriate for `X_bad`. This is closely analogous to numerical stiffness: different directions require very different effective step sizes.

Detecting non-finite values

Many failed training runs first appear as `NaN` or `inf` values. It is useful to check for them explicitly, especially while developing a new model or loss.

```
if not np.isfinite(loss):
    raise RuntimeError("non-finite loss")

if not np.isfinite(y_pred).all():
    raise RuntimeError("non-finite prediction")

if not np.isfinite(grad_theta).all():
    raise RuntimeError("non-finite gradient")
```

Non-finite values are usually symptoms. Possible causes include too large a learning rate, overflow in exponentials, division by very small numbers, incorrect scaling, or a bug in the model computation.

Classification: logits, probabilities, labels

Regression predicts continuous values. Classification predicts class membership. In binary classification, a model often produces a score called a logit:

$$z_i = x_i^T \theta + b.$$

The logit can be converted into a probability by the sigmoid function:

$$p_i = \frac{1}{1 + \exp(-z_i)}.$$

```
logits = X @ theta + bias
prob = 1.0 / (1.0 + np.exp(-logits))

labels = (prob > 0.5).astype(int)
```

The threshold 0.5 is a decision rule. The probability itself contains more information than the final label.

Stable binary cross-entropy

For binary classification, the cross-entropy loss is more appropriate than mean squared error. Written in terms of logits, it can be evaluated in a numerically stable way:

$$\ell(z, y) = \max(z, 0) - zy + \log\left(1 + \exp(-|z|)\right).$$

```python
def bce_with_logits(logits, y):
    loss_terms = (
        np.maximum(logits, 0.0)
        - logits*y
        + np.log1p(np.exp(-np.abs(logits)))
    )
    return np.mean(loss_terms)
```

This form avoids directly computing $\log(1 + \exp(z))$ for very large positive z, where overflow may occur.

Many classes: softmax

For K classes, the model may output one logit per class:

$$z_{ik}, \qquad i = 1, \dots, N, \qquad k = 1, \dots, K.$$

Softmax converts logits into class probabilities:

$$p_{ik} = \frac{\exp(z_{ik})}{\sum_{m=1}^{K} \exp(z_{im})}.$$

```
z = logits - logits.max(axis=1, keepdims=True)

log_probs = z - np.log(
    np.exp(z).sum(axis=1, keepdims=True)
)

N = logits.shape[0]
loss = -np.mean(log_probs[np.arange(N), y_class])
```

Subtracting the row maximum does not change the probabilities, but it improves numerical stability.

Regularization

Regularization modifies the loss to discourage undesirable parameter values. A common example is L^2 regularization:

$$L_{\text{reg}}(\theta) = L_{\text{data}}(\theta) + \frac{\lambda}{2}\|\theta\|^2.$$

```
lambda_reg = 1.0e-3

loss_data = 0.5 * np.mean(residual**2)
loss_reg = loss_data + 0.5 * lambda_reg*np.sum(theta**2)

grad_theta = X_train_s.T @ residual / residual.size
grad_theta += lambda_reg * theta
```

Regularization changes the optimization problem. It can improve generalization, but it also introduces a modelling choice that must be validated.

Model capacity and overfitting

A model with too little flexibility may underfit: it cannot represent the structure in the data. A model with too much flexibility may overfit: it adapts to noise or accidental details of the training set.

$$\text{underfitting} \quad \Rightarrow \quad L_{\text{train}} \text{ large}, \quad L_{\text{val}} \text{ large},$$

$$\text{overfitting} \quad \Rightarrow \quad L_{\text{train}} \text{ small}, \quad L_{\text{val}} \text{ large}.$$

The goal is not to minimize training loss at any cost. The goal is to learn a representation that remains useful outside the exact data used for fitting.

Training and validation curves

The history of training and validation losses is often more informative than the final value alone.

```python
import matplotlib.pyplot as plt

fig, ax = plt.subplots()

ax.plot(train_history, label="training")
ax.plot(val_history, label="validation")

ax.set_xlabel("step")
ax.set_ylabel("loss")
ax.set_yscale("log")
ax.legend()
```

A healthy training curve is not defined by one universal shape. However, a growing gap between training and validation losses is a common warning sign of overfitting or a mismatch between training and validation data.

→ **Example 03 - 02**

Metrics are not always losses

The loss is the scalar quantity used for optimization. A metric is a quantity used for interpretation, comparison, or reporting. They may be the same, but they do not have to be.

```
residual = y_pred - y_true

rmse = np.sqrt(np.mean(residual**2))
mae = np.mean(np.abs(residual))

accuracy = np.mean(y_pred_class == y_true_class)
```

For example, cross-entropy may be a good training loss for classification, while accuracy may be easier to communicate. In scientific problems, physically meaningful errors may be more important than generic ML scores.

Residual diagnostics

A small average error does not guarantee that the model is scientifically useful. The residual may contain systematic structure.

```
residual = y_test_pred - y_test

mean_residual = residual.mean()
rms_residual = np.sqrt(np.mean(residual**2))
max_residual = np.max(np.abs(residual))

mean_residual, rms_residual, max_residual
```

A residual should be inspected as an array, not only reduced to a scalar. If the residual depends systematically on time, position, energy, angle, or another physical variable, the model may be missing important structure.

Uncertainty and noisy data

Machine learning does not remove experimental uncertainty. If the observations have known uncertainty σ_i, this information should influence how errors are interpreted.

$$\chi^2_{\text{mean}} = \frac{1}{N} \sum_{i=1}^{N} \left(\frac{\hat{y}_i - y_i}{\sigma_i} \right)^2 .$$

```
chi2_mean = np.mean(
    ((y_pred - y_obs) / sigma)**2
)
```

A model that fits every noisy fluctuation perfectly is not necessarily better. In many scientific contexts, respecting uncertainty is more important than achieving the smallest possible training error.

Reproducibility

A training run should be reproducible enough that we can understand what changed between two results. This includes data splits, random seeds, preprocessing choices, model size, learning rate, and number of training steps.

```
rng = np.random.default_rng(123)

config = {
    "learning_rate": eta,
    "batch_size": batch_size,
    "n_epochs": n_epochs,
    "lambda_reg": lambda_reg,
    "seed": 123,
}
```

Reproducibility is not only about repeating exactly the same random sequence. It is about recording enough information to make the numerical experiment interpretable.

Scientific machine learning: residuals of equations

In scientific machine learning, the loss may include not only disagreement with data, but also disagreement with known equations. For example, if $u_\theta(t, x)$ is a model output, we may define a PDE residual

$$R_\theta(t, x) = \partial_t u_\theta - D\,\partial_x^2 u_\theta - f(u_\theta).$$

A physics-informed loss may then have several components:

$$L = L_{\text{data}} + \alpha L_{\text{PDE}} + \beta L_{\text{BC}} + \gamma L_{\text{IC}}.$$

```
loss = (
    loss_data
    + alpha*loss_pde
    + beta*loss_boundary
    + gamma*loss_initial
)
```

The weights α, β, γ are not harmless constants. They determine the relative numerical importance of different scientific constraints.

→ **Example 03 - 03**

A practical checklist

Before trusting a machine learning result, it is useful to ask the same kind of questions we ask in numerical physics.

- Do the input, target, prediction, residual, and parameter arrays have the expected shapes?
- Was preprocessing fitted only on the training data?
- Is the loss appropriate for the scientific question?
- Are the gradients finite and on a reasonable scale?
- Are training and validation diagnostics monitored separately?
- Is the result better than a simple baseline?
- Do residuals reveal systematic structure?

The main message is simple: machine learning workflows are numerical workflows. The tools are different, but the responsibility for scale, stability, validation, and interpretation remains the same.

Section 4

Further reading

Supplementary literature

- Mehta, P., Bukov, M., Wang, C.-H. *et al.* A high-bias, low-variance introduction to Machine Learning for physicists. *Physics Reports* **810**, 1–124 (2019). DOI: https://doi.org/10.1016/j.physrep.2019.03.001
- Bottou, L., Curtis, F.E., Nocedal, J. Optimization Methods for Large-Scale Machine Learning. *SIAM Review* **60**, 223–311 (2018). DOI: https://doi.org/10.1137/16M1080173
- Margossian, C.C. A review of automatic differentiation and its efficient implementation. *WIREs Data Mining and Knowledge Discovery* **9**, e1305 (2019). DOI: https://doi.org/10.1002/widm.1305
- Raissi, M., Perdikaris, P., Karniadakis, G.E. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics* **378**, 686–707 (2019). DOI: https://doi.org/10.1016/j.jcp.2018.10.045